

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen

zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli

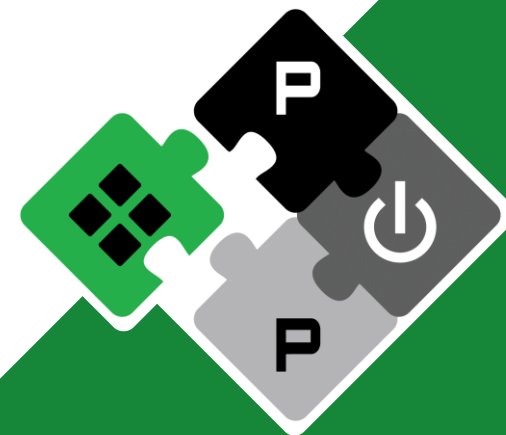
avanelli@iis.ee.ethz.ch

Luca Benini

lbenini@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!



@pulp_platform



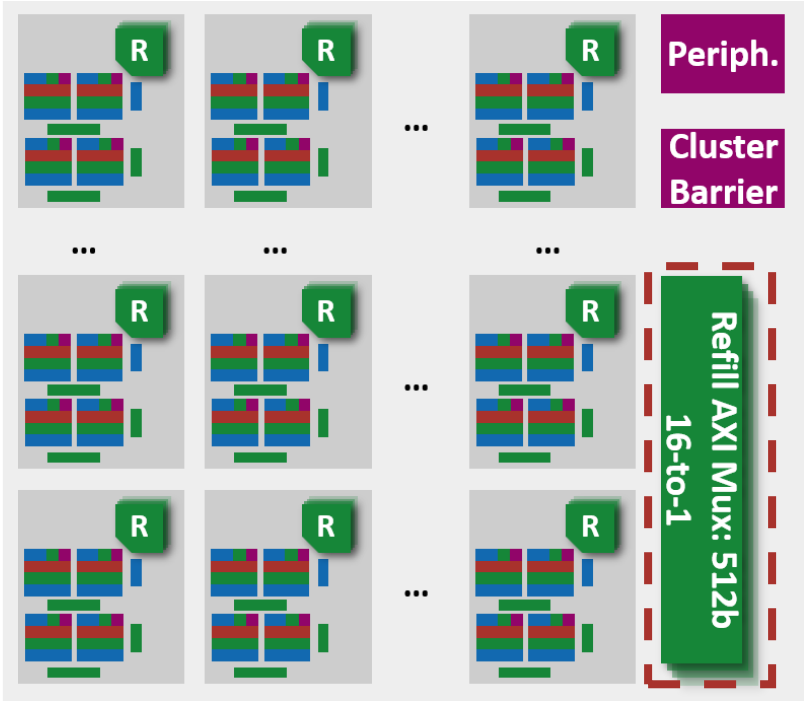
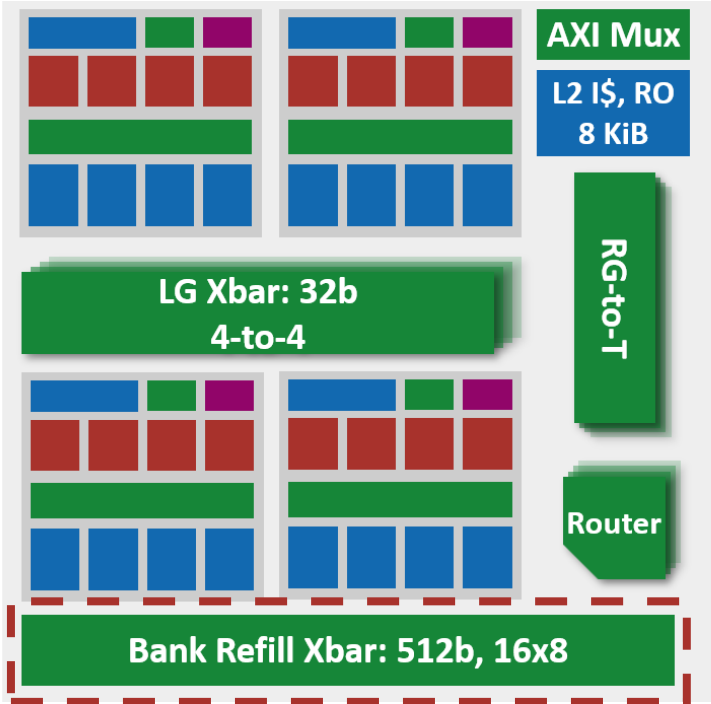
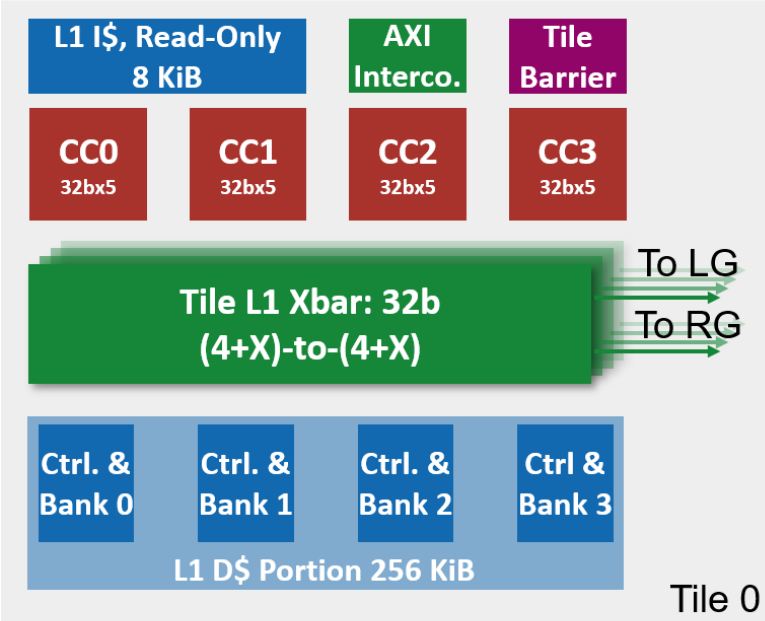
pulp-platform.org



youtube.com/pulp_platform



Hardware Development (Cluster)

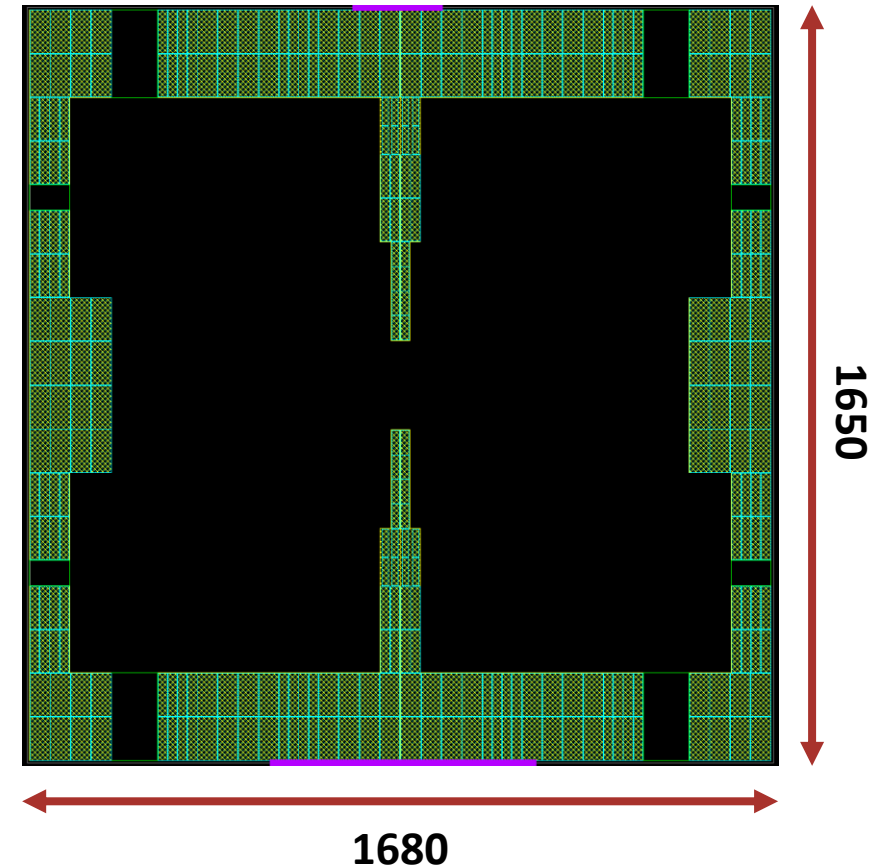
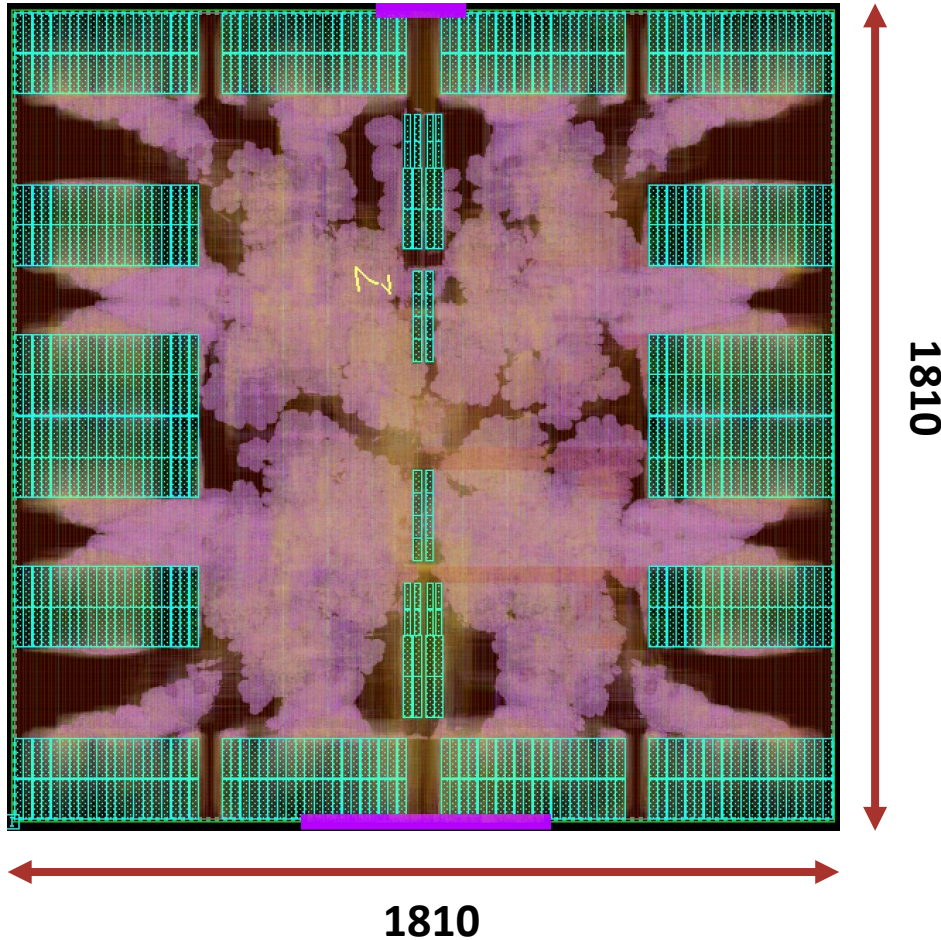


WIP: change to NoC

Hardware Development (Cluster)



Less SRAMs => FP Opt: 36% reduced
Logic Area: 2 -> 1.69 mm², 18% reduced



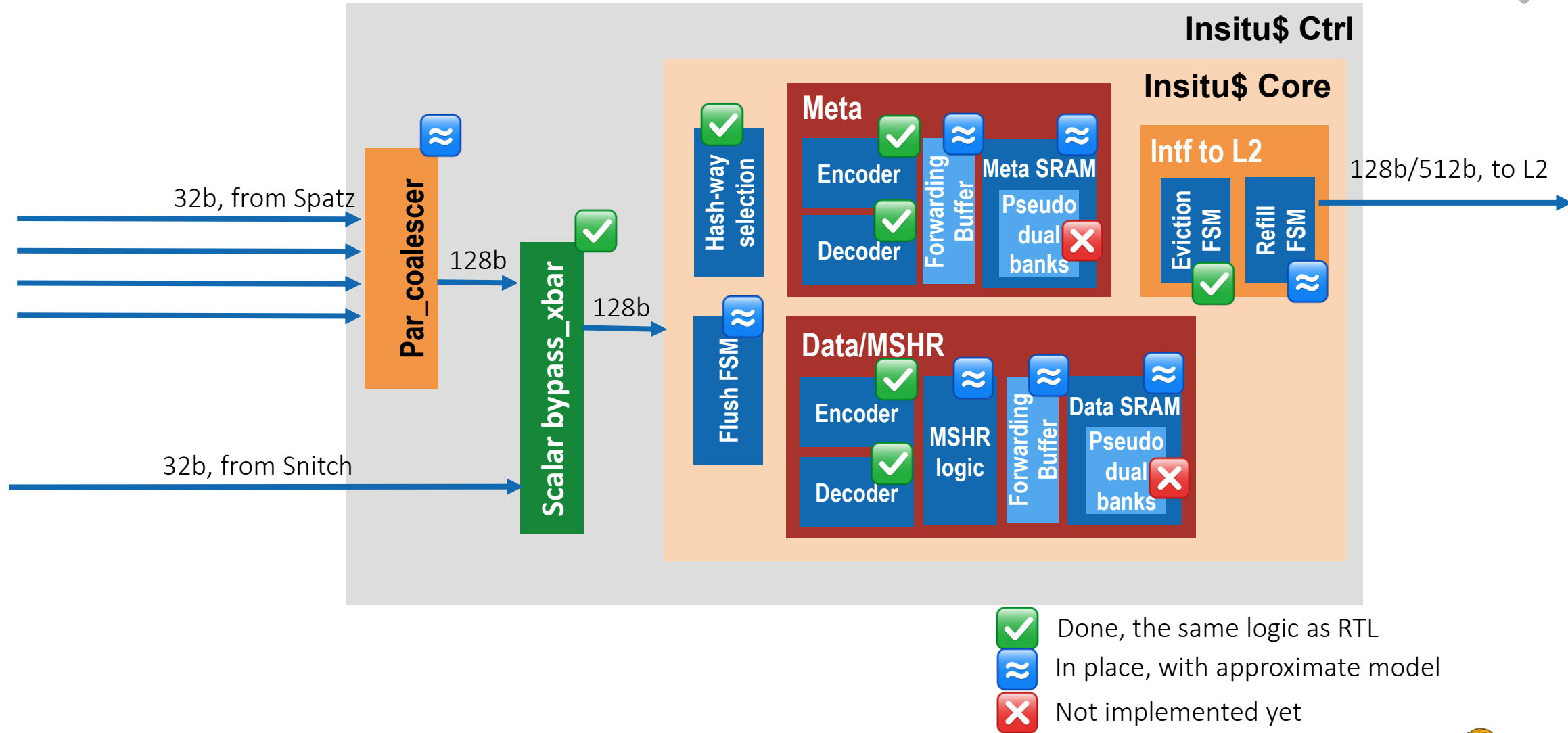
Hardware Development (Cluster)



- **Updated Block Diagram**
- **Group NoC Wrapper / Cluser**
 - Multiple bug fixes in 4x4 configurations
 - Add some AXI ID remapper => causing linting issues in SpyGlass
 - Improve CI to reduce runtime
 - More tests
- **iCache**
 - Parameterize ICache size
- **Refill NoC**
 - WIP



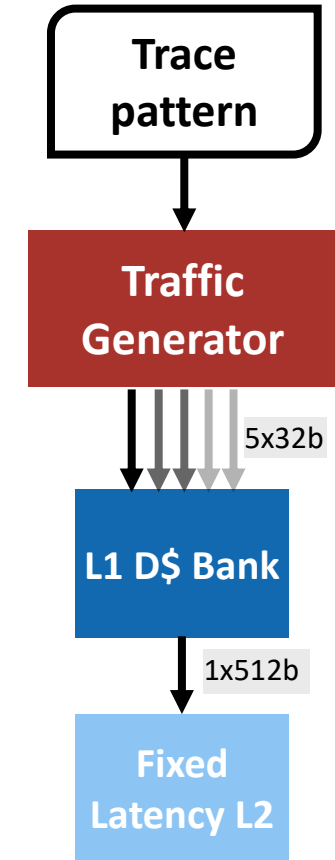
GVSoc Modeling Progress (Insitu \$)



GVSoc Modeling Progress (Insitu \$)

- Performance calibration frameworks

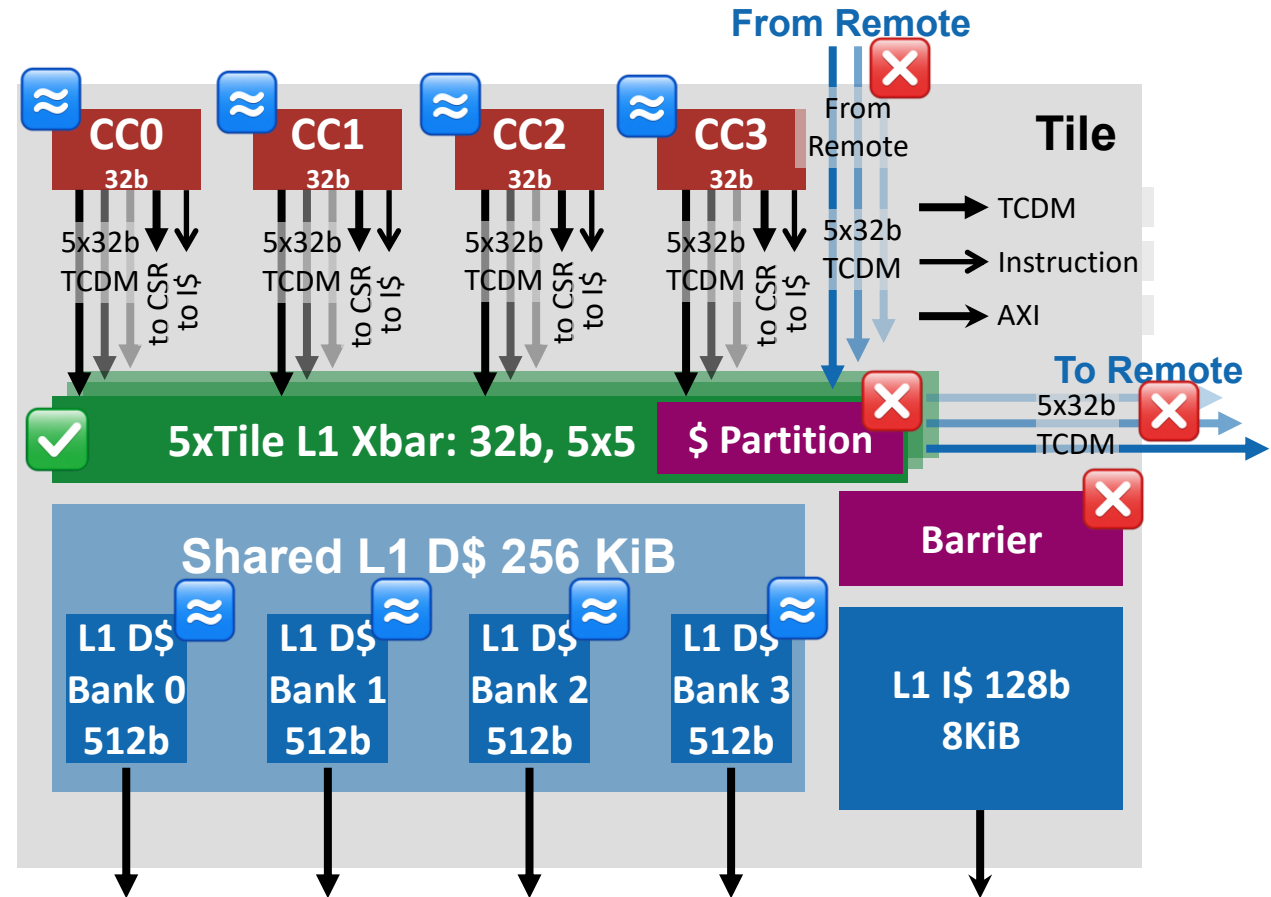
- Dedicate testbench, connect the Insitu \$ ctrl with a traffic generator and a fixed latency L2 memory
- Implement on both RTL and GVSoc side, run the same simple cases, all match or <5% diff
 - read-hit/miss latency & throughput
 - write-hit/miss latency/throughput
 - RAW (same-word) latency
 - Eviction/writeback throughput and number
 - Par-coalescer throughput (same line / different lines)
- Ongoing: Extract the cache access traces from real kernels, and feed into the standalone testbench, to see the performance difference



GVSoc Modeling Progress (Tile integration)



- Tile integration is ongoing
 - The Snitch+Spatz cc is existed
 - There is a more accurate Spatz model from a student project will be integrated later;
 - The intra-tile xbar w/ addr interleaving is implemented
 - The followings are planned
 - Cache partition logic
 - Tile-level barrier
 - Tile remote ports
 - Next: finish and the remain unimplemented components and tile integration, run real kernels for calibration



Next Step



- **InSitu-Cache Paper**

- Paper submitted to ICCD 2026. Results available on Aug 20

- **Diyou Side**

- 2 PD Run
 - Multi-group
 - Folded cache
- Debug and improve on multi-group
- Add refill NoC

- **Zexin Side**

- Continue the insitu cache and tile-level GVSoC model
- Investigate the timing report of the new multi-tile post-cts implementation, optimize the rtl is needed



Timeline



		2025							2026					
		11	12	1	2	3	4	5	6	7	8	9	10	11
System	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] Ongoing													
L1 Access Latency	Hardware optimization (Cache & Interco) [RTL] Clean up													
	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL] Complete													
Large NoC latency	Reduce per-hop latency [Physical] Ongoing by colleague													
	Cut clk-domain (optional) [RTL + Physical]													
	NoC Topologies [RTL + GVSoC + Physical] Ongoing by student													

Timeline



		2025								2026				
		11	12	1	2	3	4	5	6	7	8	9	10	11
Improve scalar core performance	Shared vector core [RTL + GVSoc] Ongoing													
SW Kernel	Kernel optimization [SW] Delayed, ongoing													
	Kernel development [SW]													
Report	Final Report and Benchmark													

L1 Access Latency



- **Problem: increased L1 access latency when scaling up**
- **Solutions**
 - **(Ongoing, RTL & Physical) Hardware optimization**
Further optimize the interconnection and cache controller to reduce access latency on existing architecture
=> Retiming the current interconnection, remove unnecessary cuts in interco and cache
=> Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
Currently has reduced to 6 cycles:
Core => Xbar => AMO => Ctrl (2) => AMO => Core
 - **(Complete, RTL) Cache partitioning**
Partition the shared L1 to keep data localized to avoid remote access latency
 - **(Complete with some problems, RTL & Physical) WT-L1**
Explore and evaluate a private WT L1 for each core
=> Performance degrade on single tile
=> Needs more work to support larger system



L1 Access Latency



- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
 - **(Ongoing, Physical) Reduce hop latency**
Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
=> Reduce the latency by 1/2
 - **(Planned, Physical) Cut clk-domain (optional)**
If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
=> Effectively cut the latency by 1/3
 - **(Planned, GVSoc & Physical) NoC Topology**
Explore different NoC topologies
=> Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2



PE-Related



- **Problem: Improve scalar core performance**

- **Solutions**

- **(Planned, RTL & GVSoc) Shared vector core**

Explore two Snitch sharing one Spatz configuration to improve scalar performance

Software complexity needs to be considered, especially on the possible conflicting use of register file.

Discussed Solution: critical section



RLC Workload



- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
 - **(Ongoing, SW) Kernel optimization**
Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
 - **(Planned, SW) Kernel development**
Implement the missing use cases for multi-user



Thank you!

Q&A

